

Lập trình .NET (ASP.NET Core Web API)

**Buổi 1: DTO, Validation,
Swagger và kiểm thử API
bằng Postman**



1. Lý thuyết

1.1. DTO

1.2. Data Annotation

1.3. Model Validation

1.4. Swagger/OpenAPI

2. Thực hành

2.1. Validate dữ liệu

2.2. Kiểm thử bằng Postman

2.3. Tùy chỉnh Swagger

DTO (Data Transfer Object) là gì?



1.1. DTO (tt)

DTO (Data Transfer Object) là gì?

Một object đơn giản chỉ chứa dữ liệu, không chứa logic nghiệp vụ.

What? DTO là một object đơn giản chỉ chứa dữ liệu, không chứa bất kỳ logic nghiệp vụ nào.

Why?

- Tránh lộ thông tin nhạy cảm của hệ thống (ví dụ: mật khẩu trong bảng User).
- Tránh vòng lặp tham chiếu (Circular Reference) khi serialize JSON.
- Giảm thiểu lượng dữ liệu truyền tải trên mạng.

When? Dùng làm đầu vào (Input/Request) hoặc đầu ra (Output/Response) của Web API.

How? Tạo class độc lập cho mục đích truyền tải.

Vấn đề khi trả về trực tiếp Entity

Nếu trả về trực tiếp `Student` Entity:

```
public class Student
```

```
{
```

```
    public int Id { get; set; }
```

```
    public string Name { get; set; }
```

```
    public string PasswordHash { get; set; } // Lộ dữ  
    liệu nhạy cảm
```

```
    public Class Class { get; set; } // Nguy cơ
```

Circular Reference

```
}
```

❑ Lộ dữ liệu nhạy cảm

PasswordHash sẽ bị serialize ra JSON và gửi tới client. Bất kỳ ai gọi API đều thấy được.

❑ Vòng lặp tham chiếu

Student.Class.Students.Class... gây lỗi khi serialize hoặc JSON phình to không cần thiết.

=> Trả về trực tiếp Student Entity qua
API - KHÔNG NÊN.

Tạo DTO riêng cho từng ngữ cảnh

Một Entity có thể có nhiều DTO - mỗi DTO phục vụ một mục đích cụ thể.

// Dùng khi tạo mới (Input)

```
public class StudentCreateDto
{
    public string Name { get; set; }
    public string Password { get; set; }
}
```

// Dùng khi trả về (Output)

```
public class StudentResponseDto
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

❑ **Không còn PasswordHash trong response.** Không còn tham chiếu vòng. Client chỉ nhận đúng dữ liệu cần thiết.

Over-posting (Mass Assignment) — lỗ hổng nguy hiểm

❑ Xảy ra khi...

Hacker gửi thêm các trường dữ liệu mà API không mong đợi — ví dụ kèm theo field **isAdmin = true** trong body request.

Ví dụ: { "name": "hacker", "email": "a@b.com", "isAdmin": true }

❑ Cách khắc phục

Sử dụng DTO chỉ chứa các thuộc tính cho phép client thay đổi. Mọi field ngoài DTO sẽ bị framework tự động bỏ qua — không bao giờ chạm tới Entity.

Các Data Annotation phổ biến

Attribute	Ý nghĩa
[Required]	Bắt buộc nhập.
[StringLength(max)]	Độ dài chuỗi tối đa.
[Range(min, max)]	Giá trị số nằm trong khoảng quy định.
[EmailAddress]	Phải là định dạng Email hợp lệ.
[RegularExpression(p)]	Phải khớp với biểu thức chính quy (Regex).

Ví dụ:

```
public class StudentCreateDto
{
    [Required(ErrorMessage = "Tên không được để trống")]
    [StringLength(50)]
    public string Name { get; set; }

    [Range(18, 100)]
    public int Age { get; set; }
}
```

1.3. Model Validation

Model Validation là gì?



1.3. Model Validation (tt)

Model Validation là gì?

□ What?

Quá trình kiểm tra dữ liệu đầu vào

Kiểm tra dữ liệu từ client **TRƯỚC** khi xử lý. Bất kỳ request nào cũng phải được xác thực trước khi đi vào business logic.

□ Why?

Đảm bảo tính toàn vẹn & hợp lệ

Ngăn chặn lỗi hệ thống và bảo mật (VD: SQL Injection, XSS). Dữ liệu bản là nguồn gốc của hầu hết các lỗ hổng web.

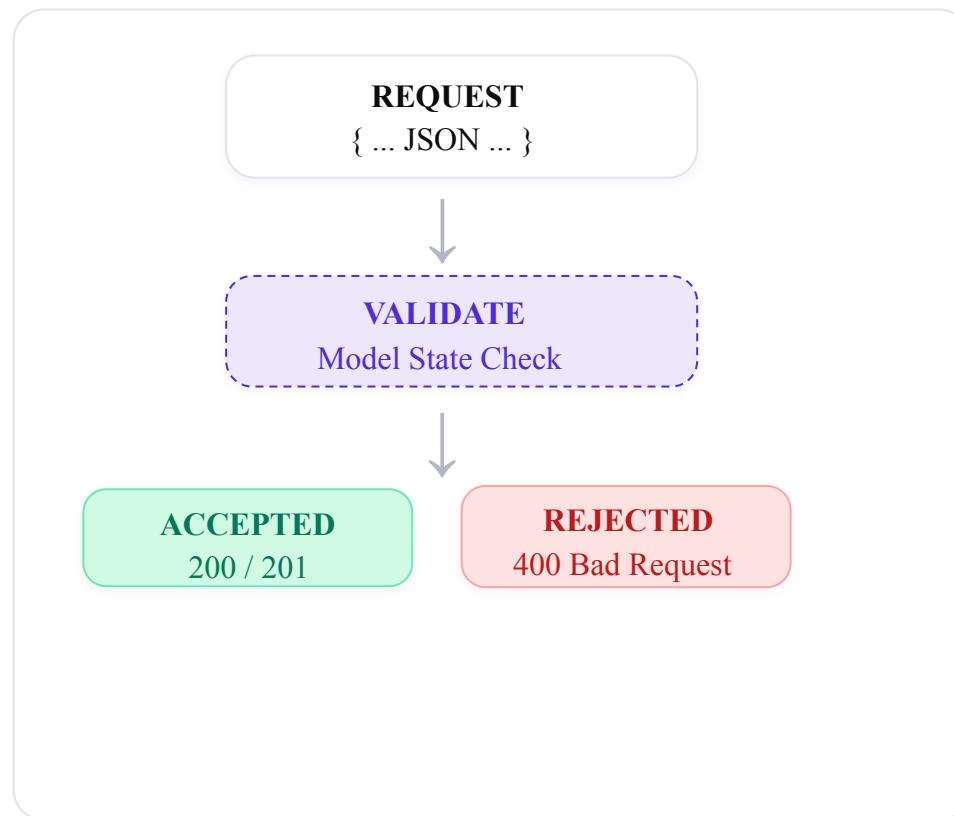
□ How?

Data Annotations trong ASP.NET Core

Sử dụng các **Attribute** đặt trên property của DTO để khai báo quy tắc kiểm tra một cách khai báo (declarative).

1.3. Model Validation (tt)

Luồng xử lý validation



Xử lý Validation trong Controller

Với [ApiController], ASP.NET Core tự động kích hoạt Model Validation.

[HttpPost]

```
public IActionResult CreateStudent([FromBody] StudentCreateDto dto)
{
    // Không cần viết: if (!ModelState.IsValid) { return BadRequest(); }
    // Framework đã lo việc đó nhờ [ApiController]

    // Xử lý tạo mới sinh viên ở đây...
    return Ok();
}
```

 ****Ghi nhớ:**** Trong ASP.NET Core, Attribute `[ApiController]` tự động kiểm tra `ModelState.IsValid`. Bạn không cần phải code thủ công phần kiểm tra này trong mỗi Action.

1.4. Swagger/OpenAPI

- **OpenAPI:** Tiêu chuẩn để mô tả các RESTful API.
- **Swagger:** Bộ công cụ xây dựng xung quanh OpenAPI Specification để thiết kế, xây dựng và làm tài liệu API.

Lợi ích:

- Tự động sinh giao diện mô tả API (Swagger UI).
- Cho phép test API trực tiếp trên trình duyệt.
- Giúp Frontend developer và Backend developer giao tiếp dễ dàng hơn.

1.4. Swagger/OpenAPI (tt)

- Có thể thêm các comment XML vào code để Swagger hiển thị mô tả chi tiết:

```
/// <summary>
```

```
/// Lấy thông tin sinh viên theo ID
```

```
/// </summary>
```

```
/// <param name="id">ID của sinh viên</param>
```

```
/// <returns>Thông tin sinh viên</returns>
```

```
[HttpGet("{id}")]
```

```
public IActionResult GetStudent(int id)
```

```
{
```

```
    // ...
```

```
}
```

Lưu ý: Cần cấu hình trong `Program.cs` và bật tùy chọn sinh file XML trong project Properties.

2.1. Kiểm thử bằng Postman

Postman là công cụ chuyên dụng để gọi và kiểm thử API.

- Thiết lập Header, Body (JSON, Form-data), Authorization
- Lưu Request thành Collections để tái sử dụng
- Tạo script test tự động
- Quản lý môi trường (Environment)

2.1. Kiểm thử bằng Postman

So sánh giữa Swagger và Postman

TIÊU CHÍ	SWAGGER	POSTMAN
Mục đích	Sinh tự động, xem tài liệu nhanh	Kiểm thử chuyên sâu, tạo test tự động
Tính năng test	Cơ bản (gửi request trên UI)	Nâng cao (script, biến, environment)
Quản lý	Không có	Collections, Environment, History
Đối tượng dùng	Dev, QA, Frontend tra cứu	QA, Backend test chuyên sâu

2.2. Demo

1. Tạo “ProductCreateDto” và “ProductResponseDto”.
2. Gắn Data Annotation validation cho các DTO.
3. Chạy API và test validate tự động bằng Swagger.
4. Sử dụng Postman để gọi POST request hợp lệ và không hợp lệ (xem JSON lỗi 400 Bad Request).
5. Tích hợp XML comments để làm đẹp Swagger.

2.3. Bài thực hành

Yêu cầu: Áp dụng DTO, Validation và kiểm thử API với Swagger, Postman.

1. Tạo “UserRegisterDto” gồm các trường: Username, Password, Email, Age.
2. Thêm Validation cho “UserRegisterDto”: Username (bắt buộc, max 20 ký tự), Password (bắt buộc, min 6 ký tự), Email (đúng chuẩn email), Age (từ 18 đến 60).
3. Cấu hình để Swagger có thể đọc được XML Comments từ các Controller và DTO. Viết summary giải thích chức năng cho một Endpoint bất kỳ.
4. Gọi Endpoint đăng ký User bằng Postman. Thử gửi Body thiếu trường Email và xem kết quả lỗi trả về.
5. Tạo một Custom Validation Attribute “[MustBeEven]” để kiểm tra một property số nguyên phải là số chẵn. Áp dụng vào một DTO.



Thank You!